
IT5429E-2-25-2 (25.2B01)

Graph Analytics for Big Data

Lecture 2: Node Embedding, Link Analysis

Many slides are adapted from <https://web.stanford.edu/class/cs224w/>

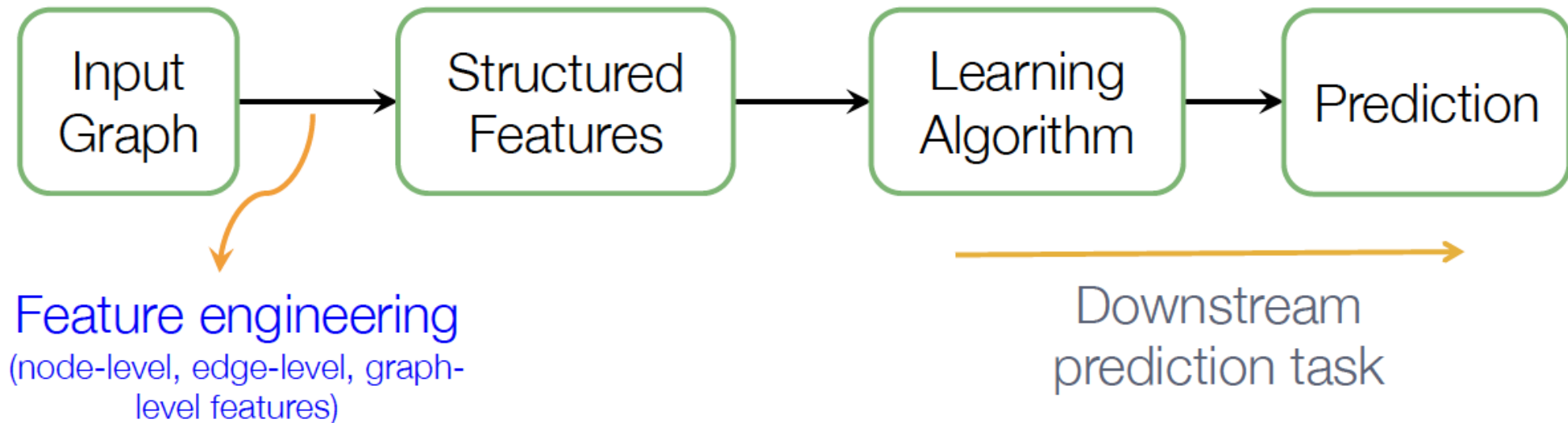


Outline

- **Graph representation learning**
 - Shallow node embedding
 - Graph embedding
 - Connection to matrix factorization
- **Link analysis**
 - PageRank
 - Personalized PageRank
 - Random walk with restarts

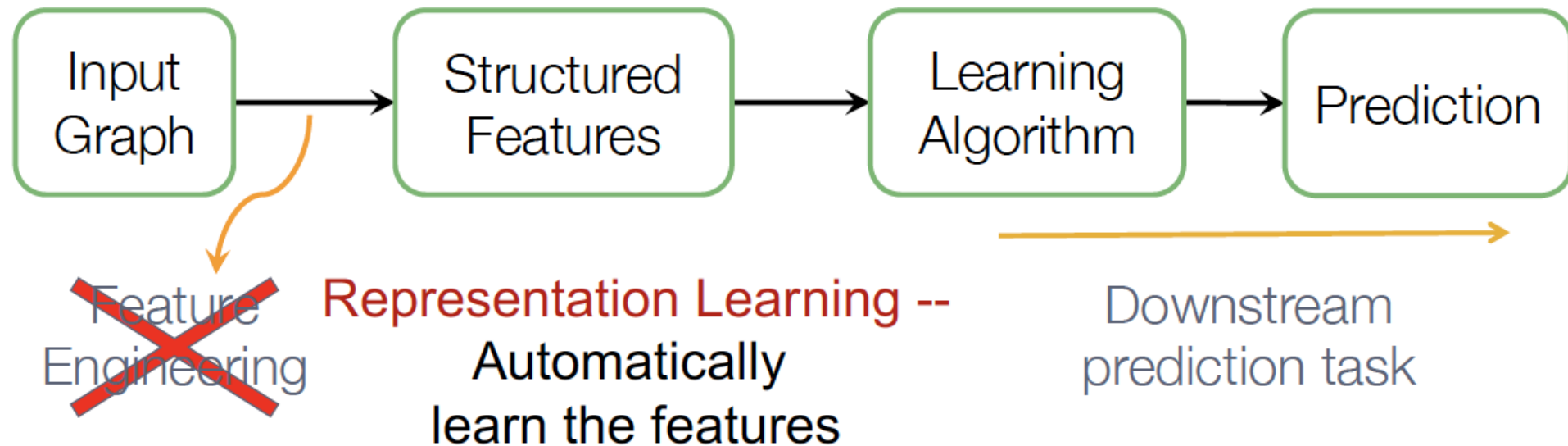
Graph Representation Learning

Recap: Traditional ML for Graphs



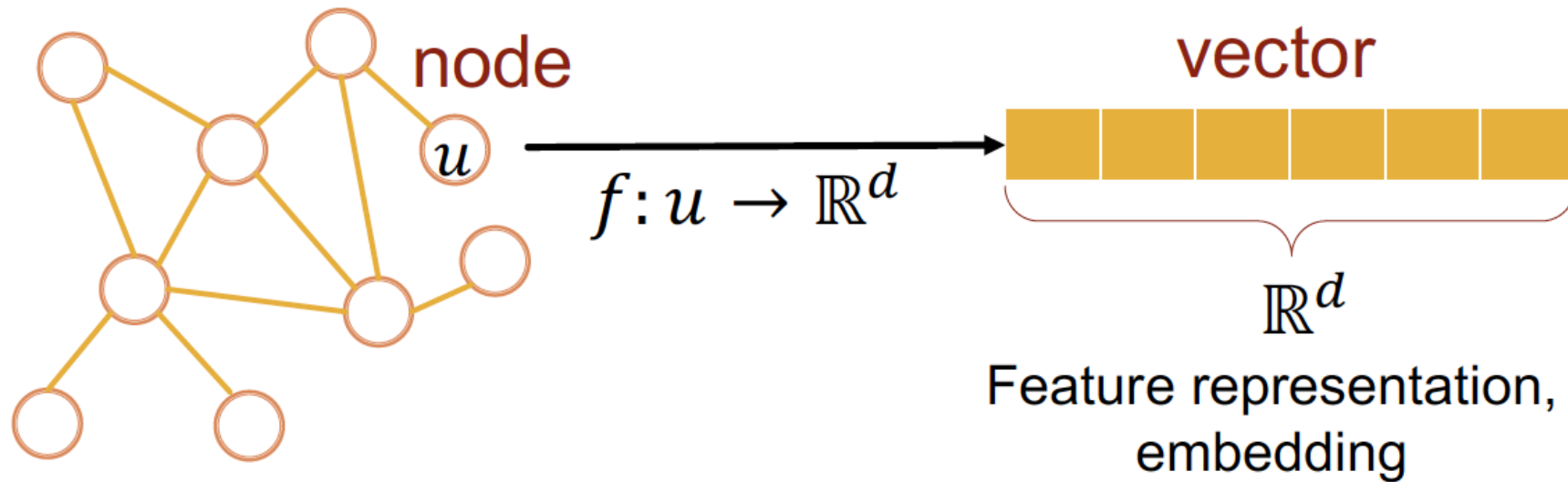
Graph Representation Learning

- Alleviate the need to do feature engineering every single time



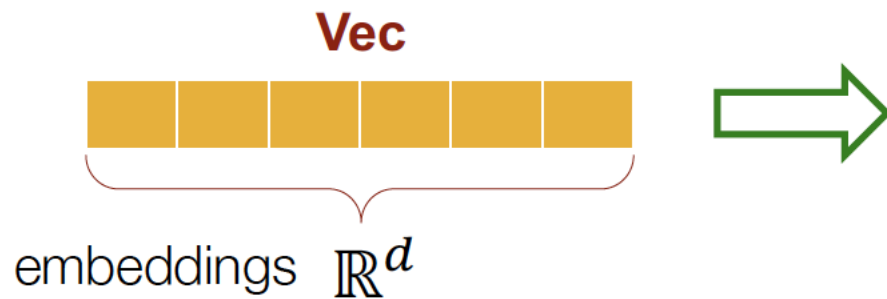
Graph Representation Learning

- **Goal:** Efficient task-independent feature learning for ML with graphs



Why Embedding

- **Task:** Map nodes into an embedding space
 - Similarity of embeddings between nodes implies their similarity in the original graph
 - Both nodes are close to each other (connected by an edge)
 - Encode graph information
 - Potentially used for many downstream predictions



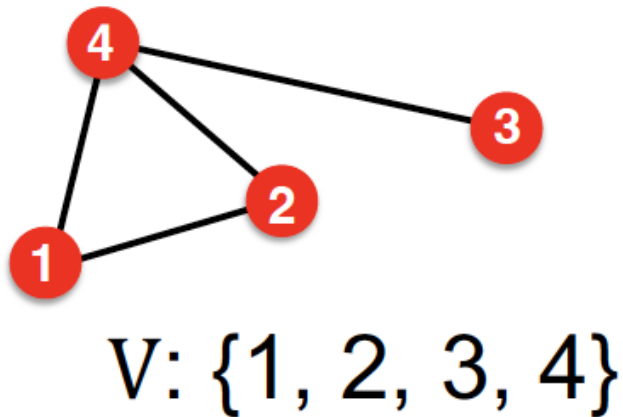
Tasks:

- Node classification
- Link prediction
- Graph classification
- Anomalous node detection
- Clustering

Node Embeddings: Encoder & Decoder

Setup

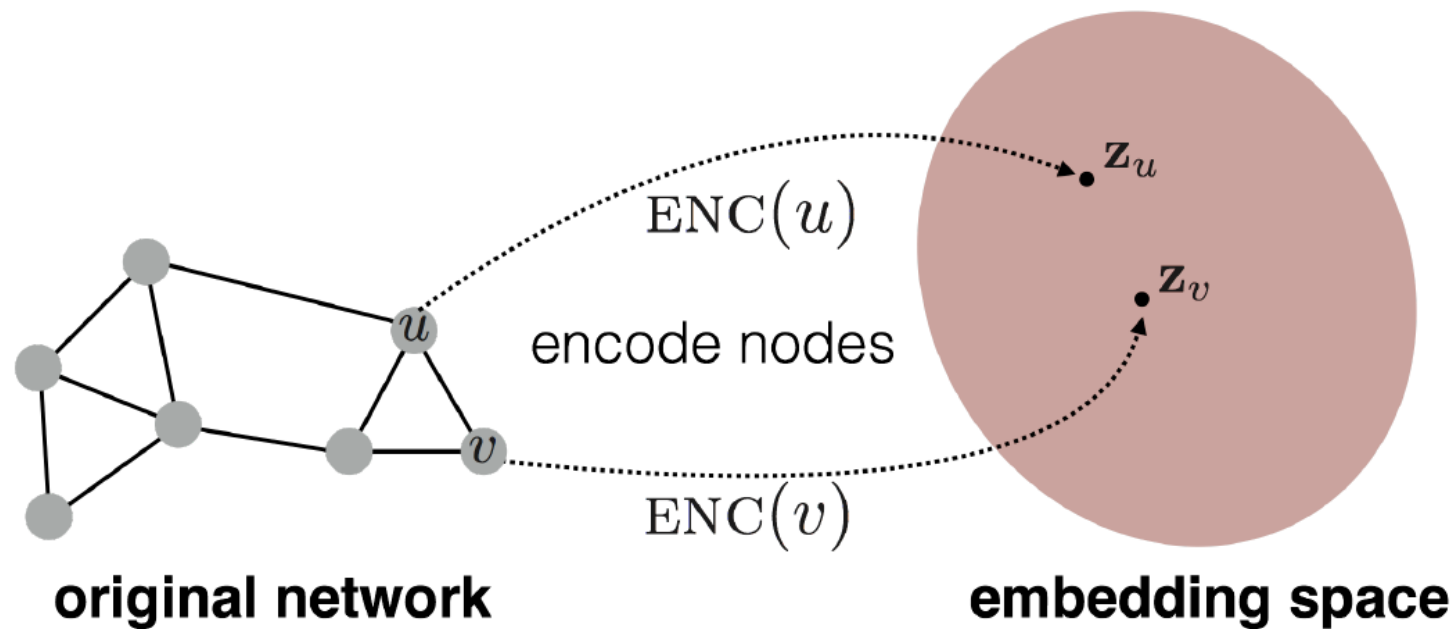
- Assume an undirected graph G
 - V : vertex set
 - A : adjacency matrix (assuming binary)
 - For simplicity: No node features or extra information



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

Embedding Nodes

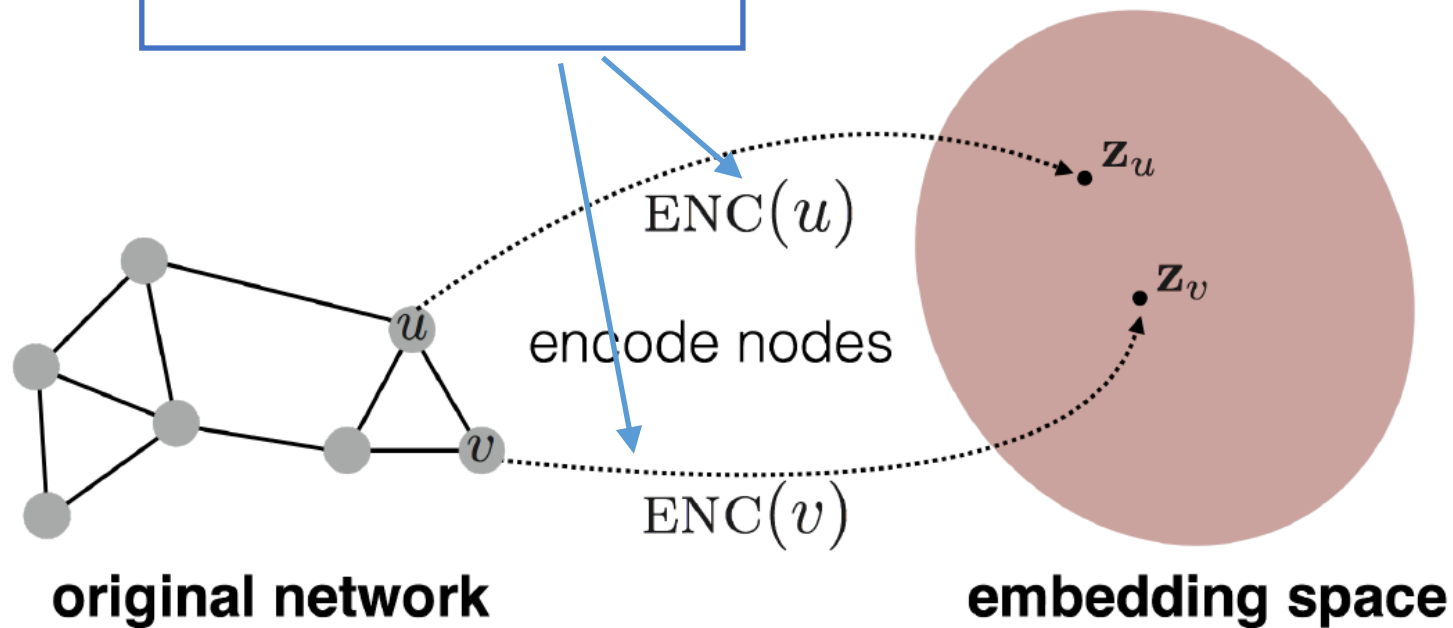
- **Goal:** encode nodes so that similarity in the embedding space (e.g., dot product) approximates similarity in graph



Embedding Nodes

- **Goal:** $\text{similarity}(u, v) \approx \mathbf{z}_u^T \mathbf{z}_v$

Need to define!!!



Learning Node Embedding

1. **Encoder** maps from nodes to embeddings
2. Define a node **similarity** function (e.g., a measure of similarity in the original graph)
3. **Decoder** (DEC) maps from embeddings to the similarity score
4. **Optimize** parameters of the encoder s.t:

$$\text{similarity}(u, v) \approx z_v^T z_u$$

in original graph

similarity of
embeddings

$$\text{DEC}(z_v^T z_u)$$

Two Key Components

- **Encoder**: map each node to a low-dimensional vector

$$\text{ENC}(v) = z_v \longleftarrow \begin{array}{l} \text{d-dimensional} \\ \text{embedding} \end{array}$$

- **Similarity function**: specify mapping of relationships between embedding space and original space

$$\text{similarity}(u, v) \approx z_v^T z_u$$

Decoder

"Shallow" Encoding

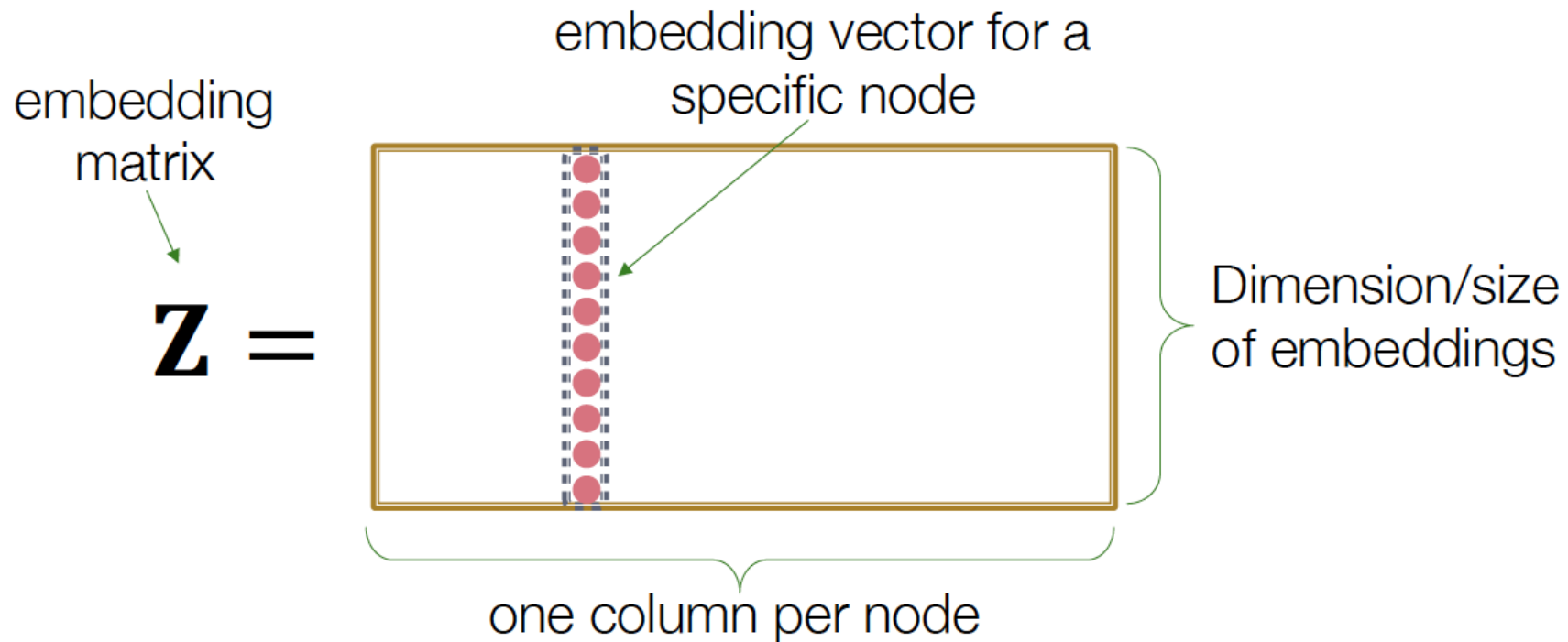
- **Simplest encoding approach:** Encoder is just an embedding-lookup

$$ENC(v) = z_v = Z \cdot v$$

- $Z \in \mathbb{R}^{d \times |V|}$: matrix, each column is a node embedding (need to optimize)
- $v \in \mathbb{I}^{|V|}$: indicator vector, all zeros except a one in column indicating node v

"Shallow" Encoding

- **Simplest encoding approach:** Encoder is just an embedding-lookup



"Shallow" Encoding

- **Simplest encoding approach:** Encoder is just an embedding-lookup

Each node is assigned a unique embedding vector (i.e., we directly optimize the embedding of each node)

- Many methods: DeepWalk, node2vec

Framework Summary

- Encoder + Decoder Framework
 - Shallow encoder: embedding lookup
 - Parameters to optimize: Z which contains node embeddings z_u for each node $u \in V$
 - We will cover deep encoders in the GNNs
- **Decoder**: based on node similarity
- **Objective**: maximize $z_v^T z_u$ for node pairs (u, v) that are **similar**.

How to Define Node Similarity

- Key choice of methods is how they define similarity
- Should two nodes have a similar embedding if they:
 - are linked?
 - share neighbors?
 - have similar “structural roles”?
- We will now learn node similarity definition that uses **random walk**, and how to optimize embeddings for such a similarity measure

Note on Node Embeddings

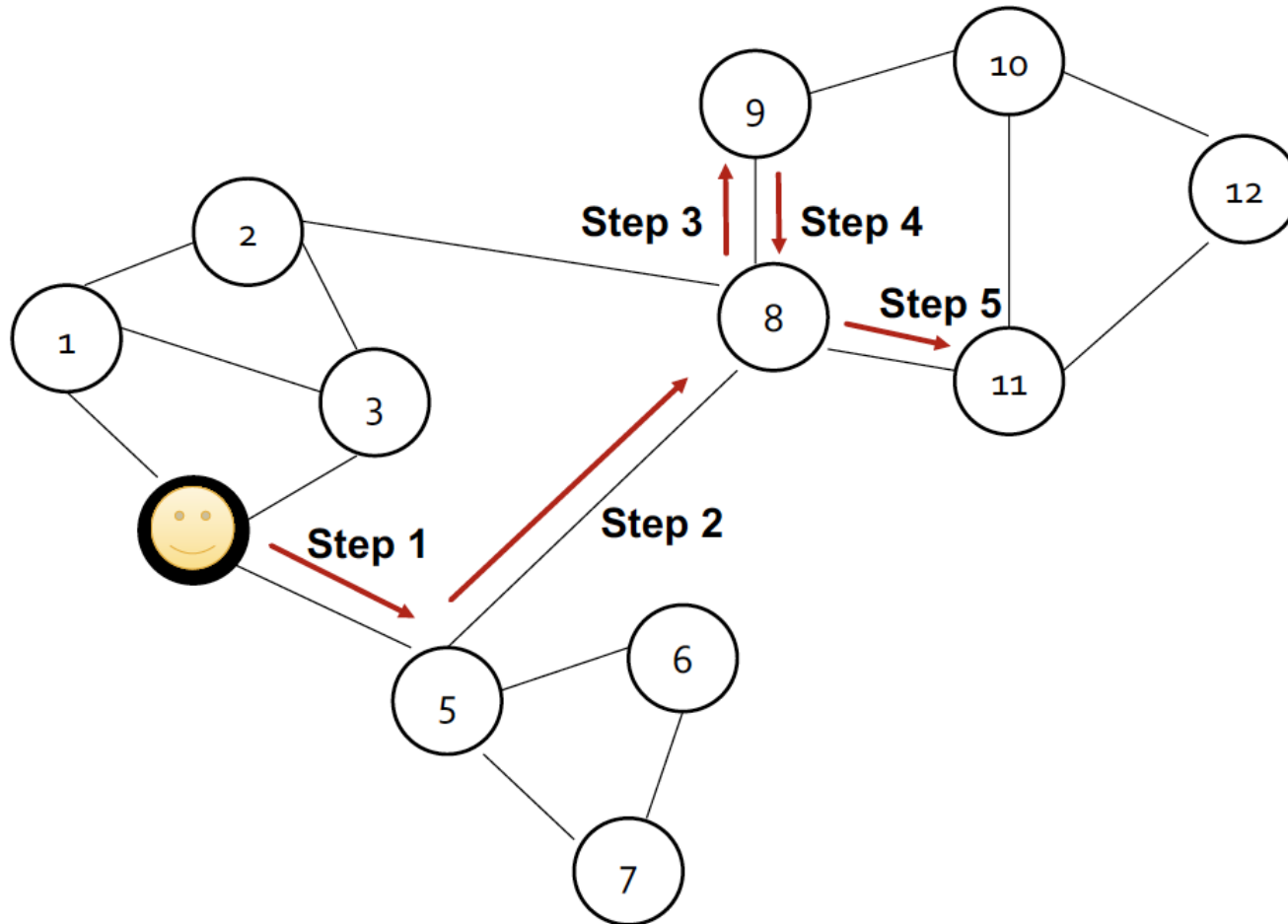
- This is unsupervised/supervised way of learning node embeddings
 - We are **not** utilizing node labels
 - We are **not** utilizing node features
 - The goal is to **directly estimate** a set of coordinates (i.e., the **embedding**) of a node so that some aspect of the network structure (captured by DEC) is preserved
- These embeddings are **task independent**
 - They are not trained for a specific task but can be used for any task

Random Walk Approach

Notations

- Vector z_u
 - Embedding of node u
- Probability $P(v \mid z_u)$:
 - Our model prediction is based on embedding z_u
 - Predicted probability of visiting node v on random walks starting from u
- Functions to predict probabilities
 - Softmax function
 - Sigmoid function

Random Walk



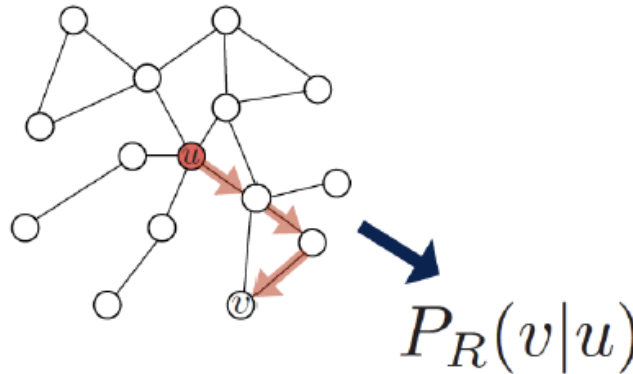
Given a graph and a starting point, we select a neighbor of it at random, and move to this neighbor; then we select a neighbor of this point at random, and move to it, etc. **The random sequence of points visited this way is a random walk on graph.**

Random-Walk Embeddings

$z_v^T z_u \approx$ Probability that u and v
co-occur on a random
walk over the graph

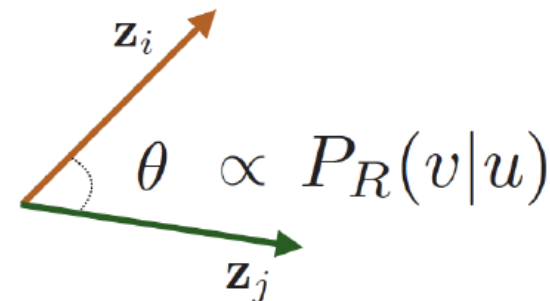
Random-Walk Embeddings

1. Estimate **probability** of visiting node v on a random walk starting from node u using some random walk **strategy** R



2. Optimize embeddings to encode these random walk statistics

Similarity in embedding space
encodes random walk similarity



Why Random Walks?

1. Expressivity:

- Flexible stochastic definition of node similarity that incorporates both local and high-order neighborhood information
- **Idea**: if random walk starting from node u visit v with **high probability**, u and v are **similar** (high-order multi-hop information)

2. Efficiency:

- Do not need to consider all node pairs when training
- Only need to consider pairs that co-occur on random walks.

Unsupervised Feature Learning

- **Intuition:** find embeddings of nodes in d-dimensional space that preserve similarity
- **Idea:** learn node embedding such that nearby nodes are close together in the graph.
- Given a node u , how do we define nearby nodes?
 - $N_R(u)$: neighborhood of u obtained by some random walk strategy R

Feature Learning as Optimization

- Given $G = (V, E)$
- Our goal is to learn a mapping $f = u \rightarrow \mathbb{R}^d: f(u) = z_u$
- Log-likelihood objective:

$$\max_z \sum_{u \in V} \log P(N_R(u) \mid z_u)$$

- Given node u , we want to learn feature representations that are predictive of the nodes in its random walk neighborhood $N_R(u)$

Random Walk Optimization

1. Run short fixed-length random walks starting from each node u in the graph using some random walk strategy R
2. For each node u collect $N_R(u)$, the multiset of nodes visited on random walks starting from u
3. Optimize embeddings according to: given node u , predict its neighbors $N_R(u)$

$$\max_z \sum_{u \in V} \log P(N_R(u) \mid z_u)$$

Random Walk Optimization

- Equivalently,

$$\min_z \mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v \mid z_u))$$

- **Intuition:** optimize embedding z_u to minimize the **negative log-likelihood** of random walk neighborhoods $N_R(u)$
- Parameterize $P(v \mid z_u)$ using softmax:

$$P(v \mid z_u) = \frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)}$$

- **Goal:** v to be most similar to u (out of all nodes n)
- **Intuition:** $\sum_i \exp(x_i) \approx \max_i \exp(x_i)$

Random Walk Optimization

- Putting it all together

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} - \log \left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)} \right)$$

Diagram illustrating the components of the loss function $\mathcal{L}$:

- $\sum_{u \in V}$: sum over all nodes u (blue box)
- $\sum_{v \in N_R(u)}$: sum over nodes v seen on random walks starting from u (pink box)
- $\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}$: predicted probability of u and v co-occurring on random walk (yellow box)

- Optimize random walk embeddings = Find embeddings that minimizes $\mathcal{L}$

Random Walk Optimization

- But doing this naively is too expensive

$$\min_z \mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)} \right)$$

Nested sum over nodes
gives $O(|V|^2)$ complexity

Random Walk Optimization

- But doing this naively is too expensive

$$\min_z \mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)} \right)$$

The normalization term from the softmax
is the culprit. Can we approximate it?

Negative Sampling

- **Solution**: negative sampling

$$-\log \left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)} \right)$$

Technically, this is a different objective. But Negative Sampling is a form of Noise Contrastive Estimation (NCE) which approx. maximizes the log probability of softmax.

Reference: <https://arxiv.org/pdf/1402.3722>

$$\approx \log(\sigma(z_u^T z_v)) + \sum_{i=1}^k \log(\sigma(-z_u^T z_{n_i})), n_i \sim P_V$$

Sigmoid function

Random distribution
over nodes

- Just normalize against k random **negative** samples n_i
 - Quick neighborhood calculation

Negative Sampling

- Sampling k negative nodes n_i , each with prob. proportional to its degree
- Two considerations for k :
 - Higher k gives more robust estimates
 - Higher k corresponds to higher bias on negative events.
 - Typical choice: $k = 5$ to 20
- Can negative samples be any node or only nodes not on the walk?
 - People often sample any nodes (for efficiency)

Stochastic Gradient Descent

- Evaluate gradient for each individual training example:
 1. Initialize z_u at some randomized value for all nodes u
 2. Iterate until convergence: $\mathcal{L}^{(u)} = \sum_{v \in N_R(u)} -\log(P(v \mid z_u))$
 - Sample a node u , for all v calculate the gradient $\frac{\partial \mathcal{L}^{(u)}}{\partial z_v}$
 - For all v , update: $z_v \leftarrow z_v - \eta \frac{\partial \mathcal{L}^{(u)}}{\partial z_v}$

Random Walk Summary

1. Run short fixed-length random walks starting from each node in the graph
2. For each node u , collect $N_R(u)$, the multi-set of nodes visited on random walks starting from u
3. Optimize embeddings Z using Stochastic Gradient Descent:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(z_u^T z_v)}{\sum_{n \in V} \exp(z_u^T z_n)} \right)$$

We can efficiently
approximate this using
negative sampling

How Should We Randomly Walk?

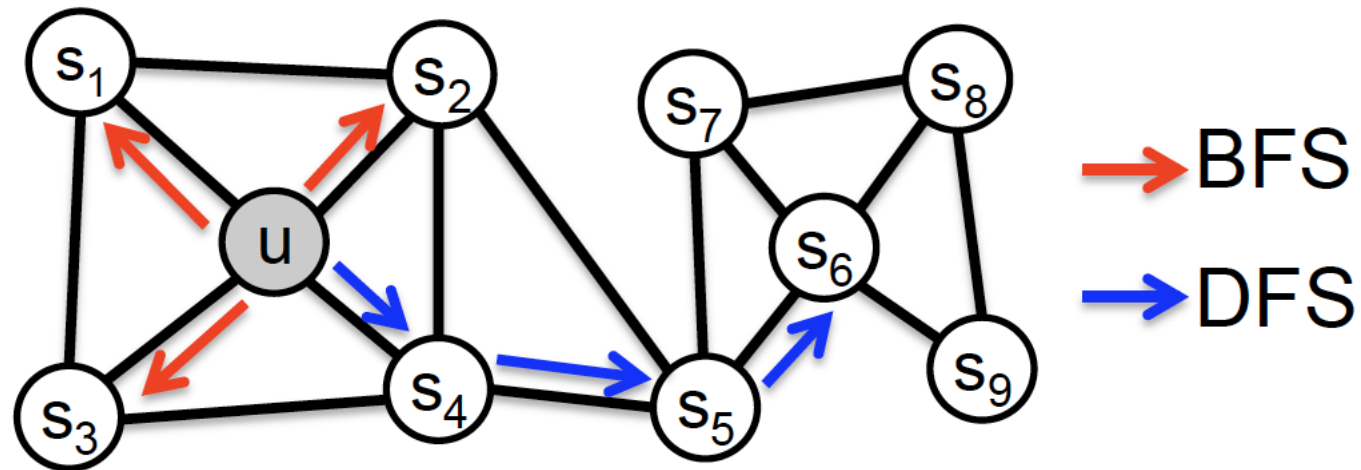
- What strategies should we use to run these random walks?
 - **Simplest idea:** Just run fixed-length, unbiased random walks starting from each node ([DeepWalk from Perozzi et al., 2013](#))
 - The issue is that such notion of similarity is too constrained.
- How can we generalize this?

Overview of node2vec

- **Goal:** Embed nodes with **similar** network neighborhoods **close** in embedding space.
- We frame this goal as a maximum likelihood optimization problem, independent to the downstream prediction task.
- **Key observation:** Flexible notion of network neighborhood $N_R(u)$ of node u leads to rich node embeddings
- Develop biased random walk R to generate network neighborhood $N_R(u)$ of node u .

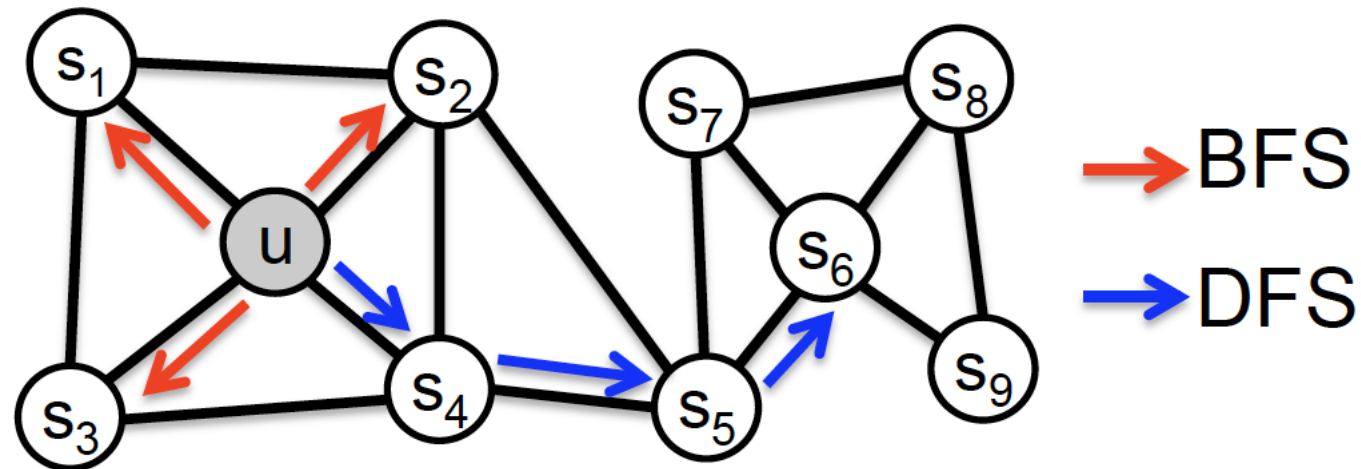
node2vec: Biased Walks

- **Idea:** use flexible, biased random walks that can trade off between **local** and **global** views of the network ([Grover and Leskovec, 2016](#))



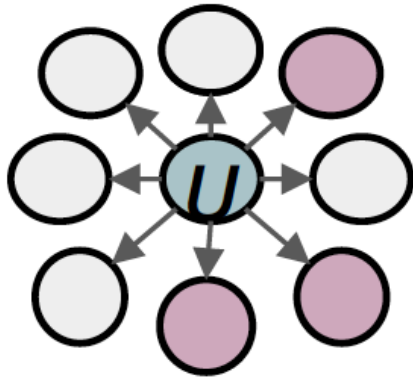
node2vec: Biased Walks

- Two classic strategies to define a neighborhood $N_R(u)$ of a node u .

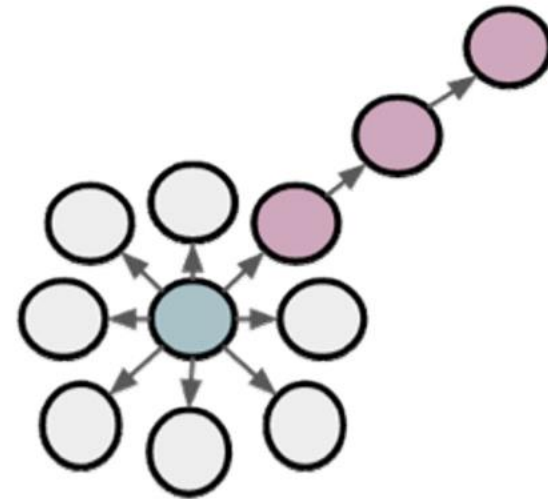


- Walk of length 3 ($N_R(u)$ of size 3)
 - $N_{BFS}(u) = \{s_1, s_2, s_3\}$: Local microscopic view
 - $N_{DFS}(u) = \{s_4, s_5, s_6\}$: Global macroscopic view

BFS versus DFS



BFS: $N_R(\cdot)$ will provide a
micro-view of
neighborhood



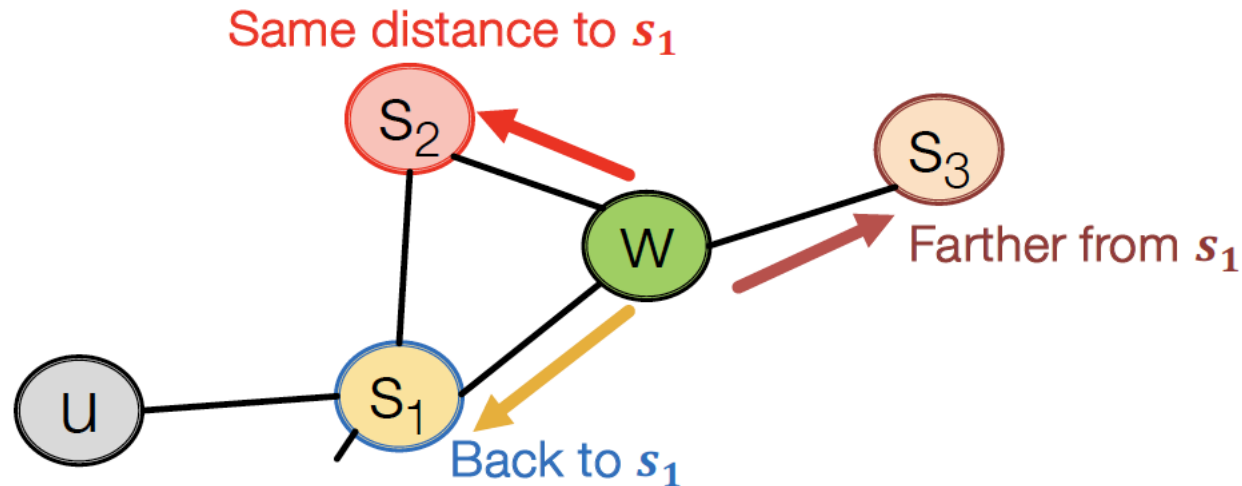
DFS: $N_R(\cdot)$ will provide a
macro-view of
neighborhood

Interpolating BFS and DFS

- Biased fixed-length random walk R that generates neighborhood $N_R(u)$ for a given node u .
- Random walk has two parameters:
 - **Return parameter p :**
 - Return to previous node
 - **In-out parameter q**
 - Moving outwards (DFS) vs inwards (BFS) from previous node
 - Intuitively, q is the ratio of BFS vs DFS
- Next, we specify how a single step of biased random walk is performed
- Random walk is a sequence of these steps

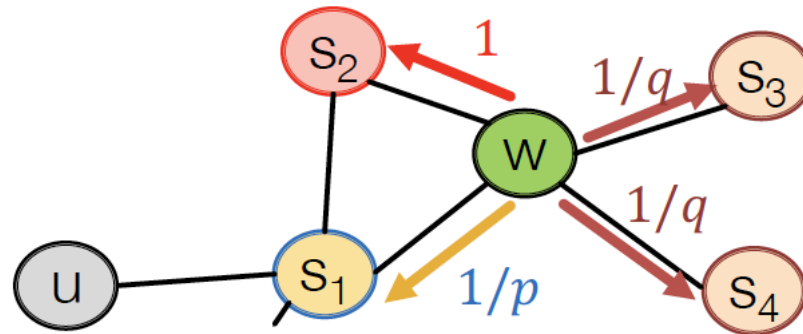
One Step of Biased Random Walk

- Define the random walk by specifying the walk **transition probabilities** on edges adjacent to the current node w
 - Random walk just traversed edge (s_1, w) and is now at w .
 - We specify edge transition probabilities out of node w
 - Insight:** neighbors of w can only be:



One Step of Biased Random Walk

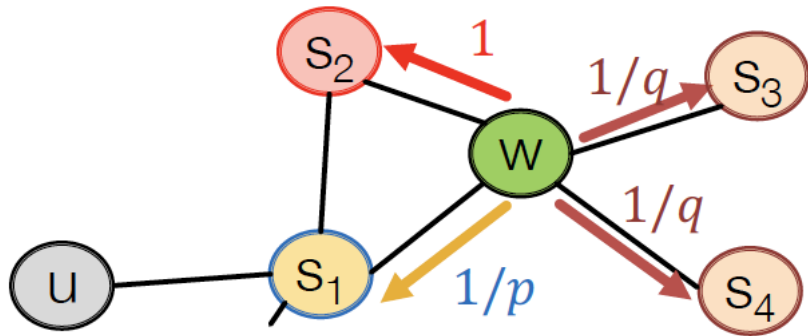
- Walker came over edge (s_1, w) and is now at w . How to set edge transition probabilities?



- p, q model transition probabilities:
 - p : return parameter
 - q : walk-away parameter

One Step of Biased Random Walk

- Walker came over edge (s_1, w) and is now at w . How to set edge transition probabilities?



$N_R(u)$ are the nodes visited by biased walk

- BFS-like walk: low value of p
- DFS-like walk: low value of q

node2vec Algorithm

1. Compute edge transition probabilities
 - For each edge (s_1, w) we compute edge walk probabilities (based on p and q) of edges $(w, \cdot)$
 2. Simulate r random walks of length l starting from each node u
 3. Optimize node2vec objective using stochastic gradient descent
-
- Linear-time complexity
 - All 3 steps are individually parallelizable

Other Random Walk Ideas

- Different kinds of biased random walks:
 - Based on node attributes ([Dong et al., 2017](#))
 - Based on learnt weights ([Abu-El-Haija et al., 2017](#))
- Alternative optimization schemes
 - Directly optimize based on 1-hop and 2-hop random walk probabilities (LINE from [Tang et al. 2015](#))
- Network preprocessing techniques
 - Random walks on modified versions of original graph ([Ribeiro et al. 2017's struct2vec](#), [Chen et al. 2016's HARP](#))

Summary

- **Core idea:** embed nodes so that distances in embedding space reflect node similarities in the original graph
- Different notions of node similarity
 - Naïve: similar if two nodes are connected
 - Random walk approaches

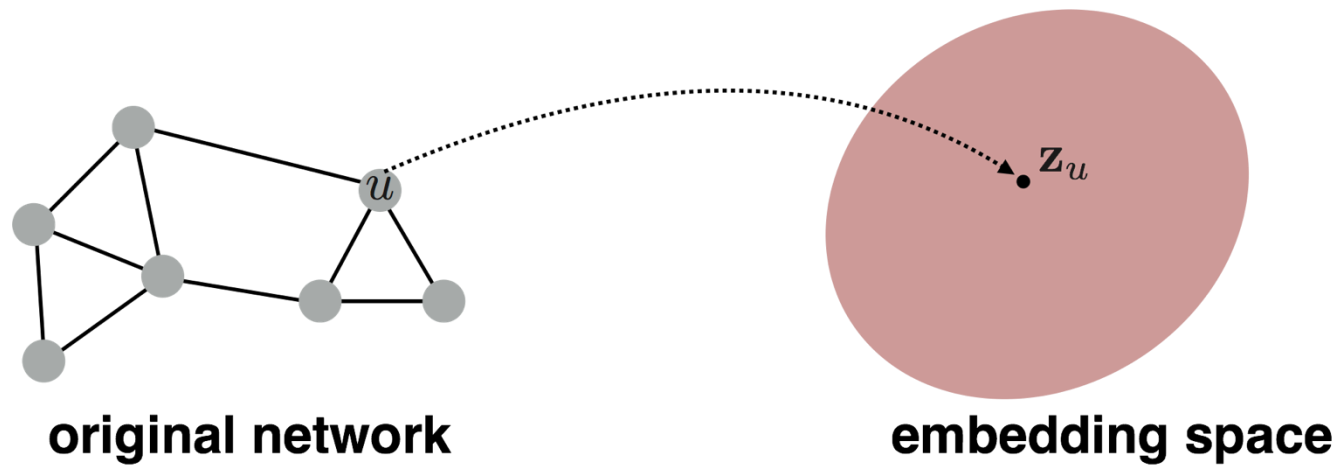
Summary

- No method wins in all cases
 - E.g., node2vec performs better on node classification while alternative methods perform better on link prediction ([Goyal and Ferrara, 2017 survey](#))
- Random walk approaches are generally more efficient
- In general, must choose definition of node similarity that matches your application

Embedding Entire Graphs

Embedding Entire Graphs

- **Goal:** want to embed a subgraph or an entire graph G . Graph embedding: z_G



- **Tasks:**
 - Classifying toxic versus non-toxic molecules
 - Identifying anomalous graphs

Approach 1

Simple but effective approach

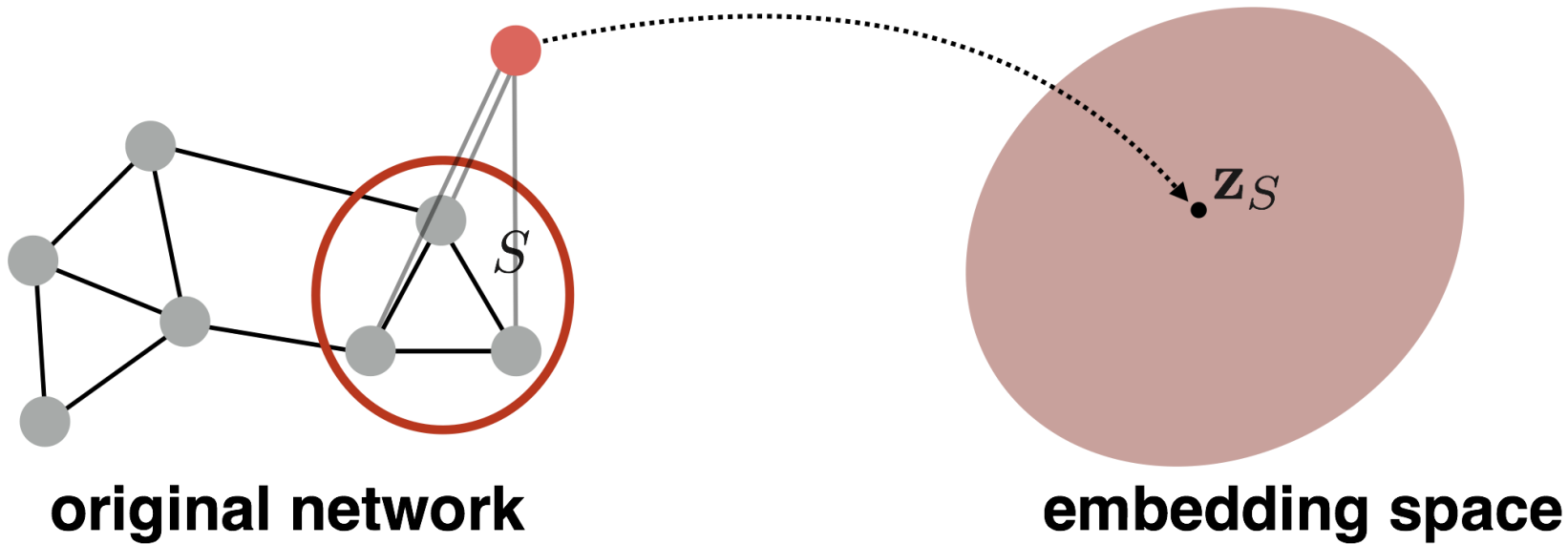
- Run a standard node embedding technique on a (sub)-graph G
- Then just **sum (or average) the node embeddings** in the sub-graph G :

$$z_G = \sum_{v \in G} z_v$$

- Used by [Duvenaud et al., 2016](#) to classify molecules based on their graph structure

Approach 2

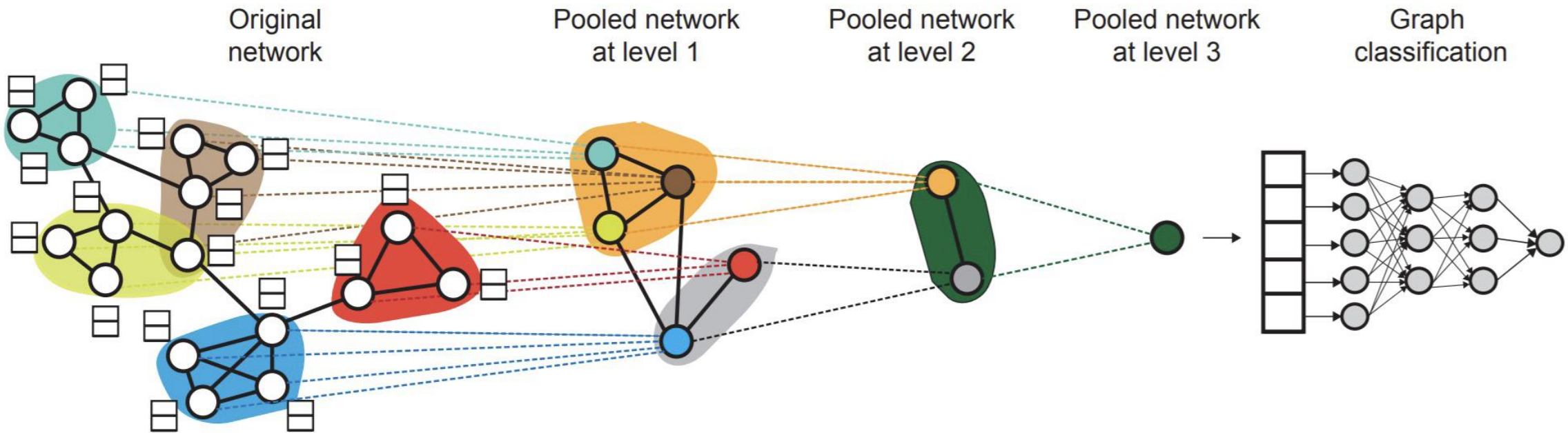
- Introduce a **virtual node** to represent the sub-graph and run the standard node embedding technique



- Proposed by [Li et al., 2016](#) as a general technique for subgraph embedding

Preview: Hierarchical Embeddings

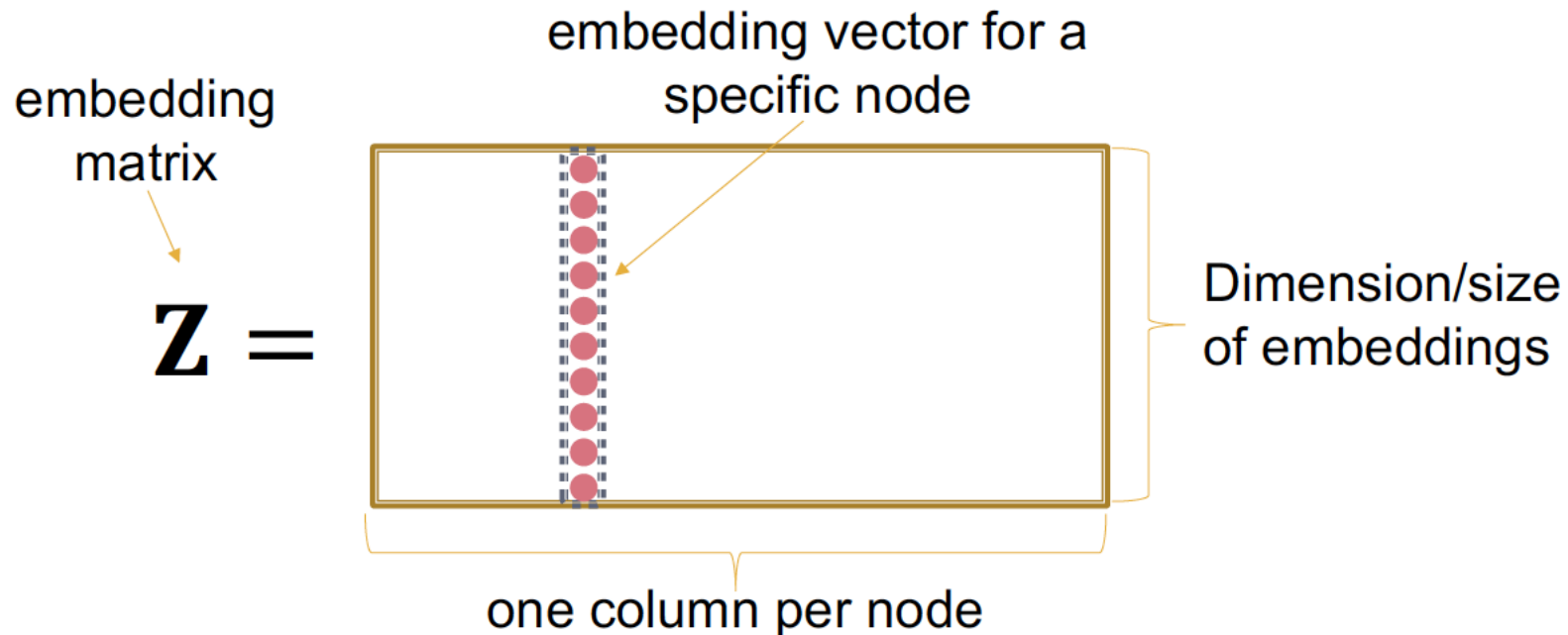
- **DiffPool**: we can also **hierarchically cluster nodes** in graphs, and **sum/avg** the node embeddings according to these clusters



Matrix Factorization and Node Embeddings

Embeddings

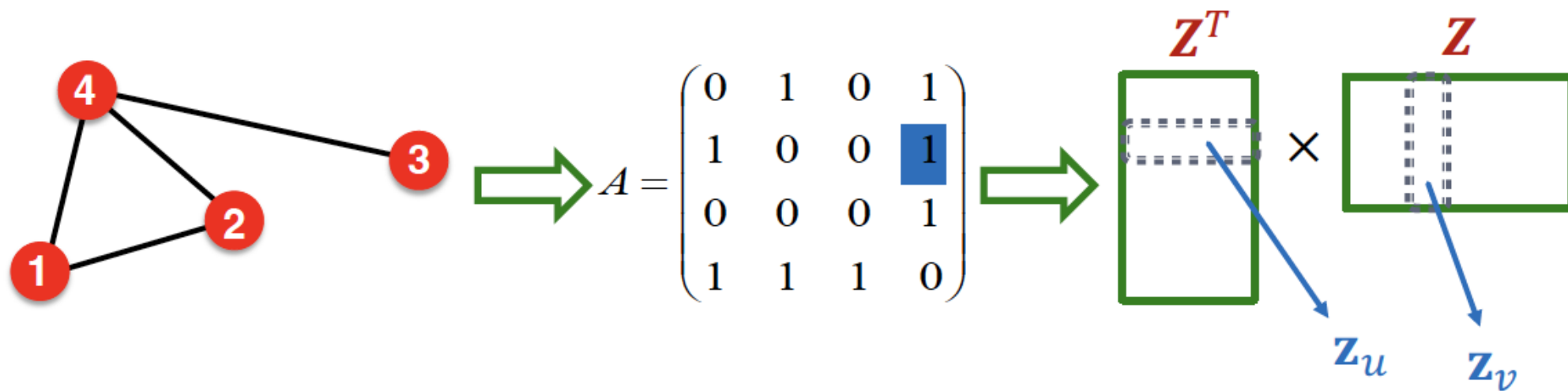
- **Recall:** encoder as an embedding lookup



- **Objective:** maximize $z_v^T z_u$ for node pairs (u, v) that are similar

Connection to Matrix Factorization

- **Simplest node similarity**: node u, v are similar if they are connected by an edge.
- This means: $z_v^T z_u = A_{u,v}$ which is the (u, v) entry of the adjacency matrix A
- Therefore, $Z^T Z = A$



Matrix Factorization

- The embedding dimension d (number of rows in Z) is much smaller than $\#nodes$ n
- Exact factorization $A = Z^T Z$ is generally not possible
- However, we can learn Z approximately
- **Objective:** $\min_Z \|A - Z^T Z\|_2$
 - We optimize Z to minimize the L2 norm of $A - Z^T Z$
 - Note today we used softmax instead of L2. But the goal to approximate A with $Z^T Z$ is the same.
- **Conclusion:** inner product decoder with node similarity defined by edge connectivity is **equivalent** to matrix factorization of A .

Random Walk-Biased Similarity

- DeepWalk and node2vec have a more complex node similarity definition based on random walks
- DeepWalk is equivalent to matrix factorization of the following complex matrix expression:

$$\log \left(\text{vol}(G) \left(\frac{1}{T} \sum_{r=1}^T (D^{-1}A)^r \right) D^{-1} \right) - \log b$$

Random Walk-Biased Similarity

Volume of graph

$$vol(G) = \sum_i \sum_j A_{i,j}$$

Diagonal matrix D
 $D_{u,u} = \deg(u)$

$$\log \left(vol(G) \left(\frac{1}{T} \sum_{r=1}^T (D^{-1}A)^r \right) D^{-1} \right) - \log b$$

context window size
 $T = |N_R(u)|$

Power of normalized adjacency matrix

Number of negative samples

- Node2vec can also be formulated as a matrix factorization (albeit a more complex matrix)

How to Use Embeddings

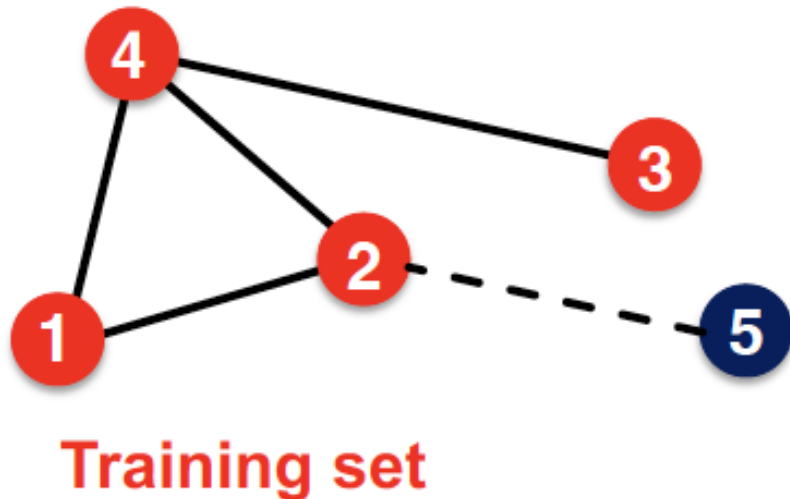
- Clustering/community detection
- Node classification
- Link prediction: predict edge (i, j) based on (z_i, z_j)
 - We can concatenate, avg, product or subtract between the embeddings
 - Concatenate: $f(z_i, z_j) = g([z_i, z_j])$
 - Hadamard: $f(z_i, z_j) = g(z_i * z_j)$
 - Sum/avg: $f(z_i, z_j) = g(z_i + z_j)$
 - Distance: $f(z_i, z_j) = g(\|z_i - z_j\|_2)$
- Graph classification: predict labels based on graph embedding z_G via aggregating node embeddings or virtual node

Summary

- We discussed graph representation learning, a way to learn node and graph embeddings for downstream tasks, without feature engineering.
- Encoder-decoder framework:
 - Encoder: embedding lookup
 - Decoder: predict score based on embedding to match node similarity
- Node similarity measure: (biased) random walk
 - Example: DeepWalk, node2vec
- Extension to graph embedding:
 - Node embedding aggregation

Limitations

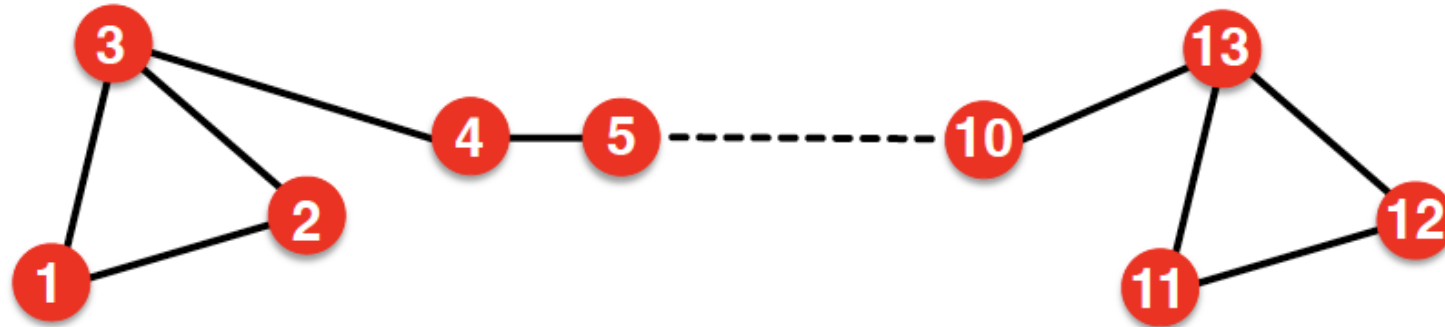
- Transductive (not inductive) method: cannot obtain embeddings for node not in the training set. Cannot apply to new graphs, evolving graphs



A newly added node 5 at test time (e.g., new user in a social network). Cannot compute its embedding with DeepWalk/node2vec. Need to recompute all node embeddings

Limitations

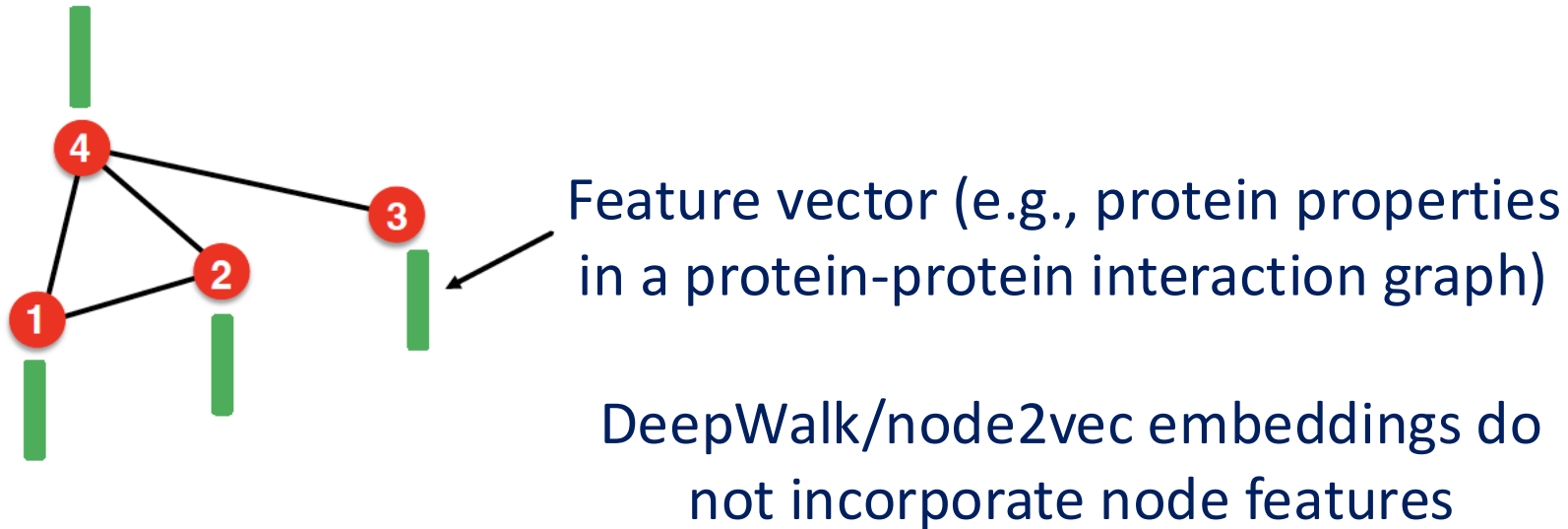
- Cannot capture structural similarity



- Node 1 and 11 are structurally similar – part of one triangle, degree 2, etc.
- However, they have very different embeddings
 - It is unlikely that a random walk will reach node 1 from node 11.

Limitations

- Cannot utilize node, edge, and graph features

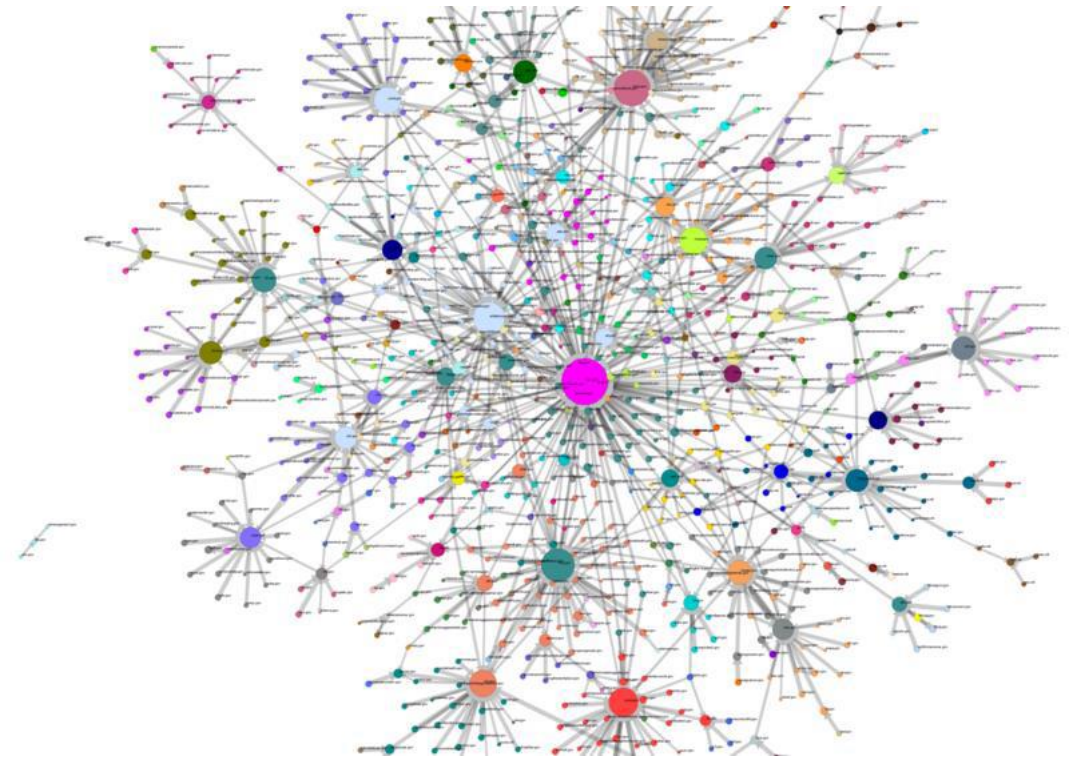


- **Solution:** Deep Representation Learning with Graph Neural Networks

Link Analysis and PageRank (Google Algorithm)

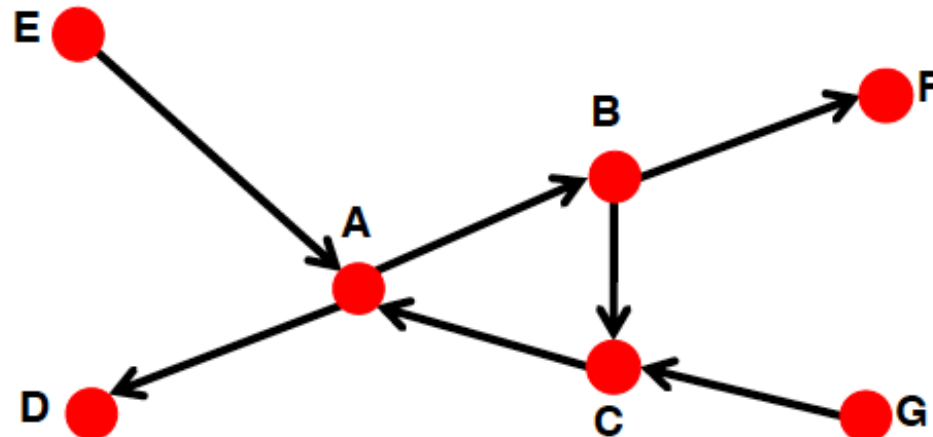
Example: The Web as a Graph

- Web as a graph
 - Nodes = web pages
 - Edges = hyperlinks
- Side issue: What is a node
 - Dynamic pages created on the fly
 - “Dark matter” -- inaccessible database generated pages



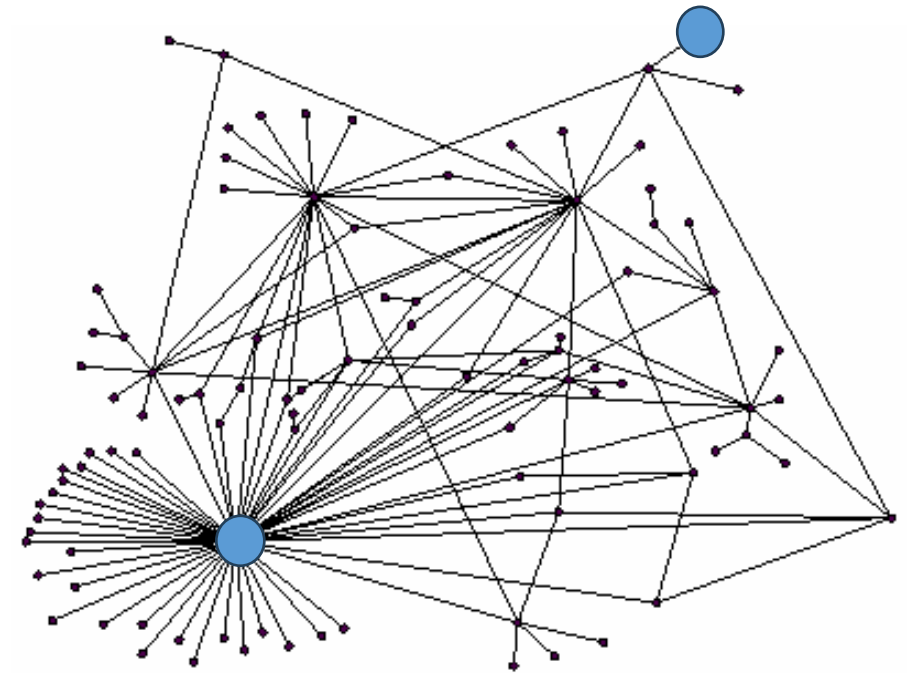
What does The Web Look Like?

- How is the Web linked?
- What is the map of the Web?
- Web as a directed graph:
 - Given a node v , what nodes can v reach?
 - What other nodes can reach v ?



Ranking Nodes on the Graph

- All webpages are not equally important
- There is a large diversity in the web-graph node connectivity
- **Goal:** rank the pages using the web graph link structure



Link Analysis Algorithms

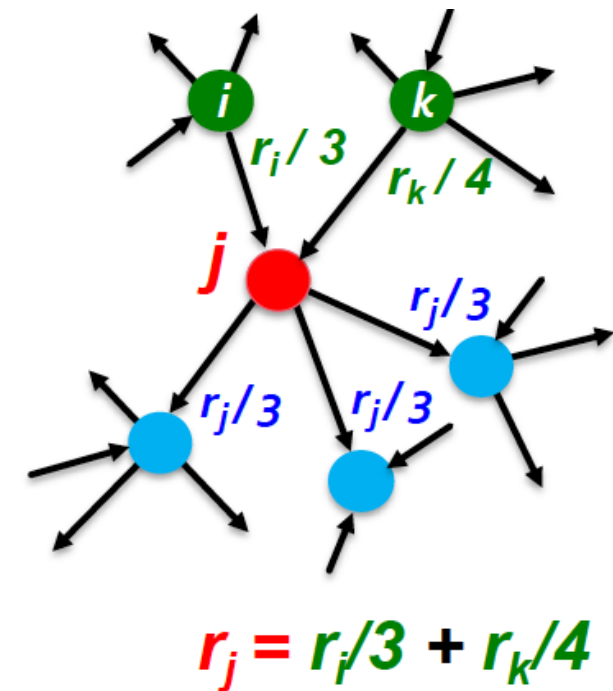
- Compute the importance of nodes in a graph
 - PageRank
 - Personalized PageRank (PPR)
 - Random Walk with Restarts

Links as Votes

- **Ideas:** Links as votes
 - Page is more important if it has more links
 - In-coming links? Out-coming links?
- Think of in-links as votes
- Are all in-links equal?
 - Links from important pages count more
 - Recursive question!!!

PageRank: The "Flow" Model

- A vote from an important page is worth more:
 - Each link's vote is proportional to the importance of its source page
 - If page i with importance r_i has d_i out-links, each out-link gets $\frac{r_i}{d_i}$ votes
- Page j 's own importance r_j is the sum of votes on its in-links

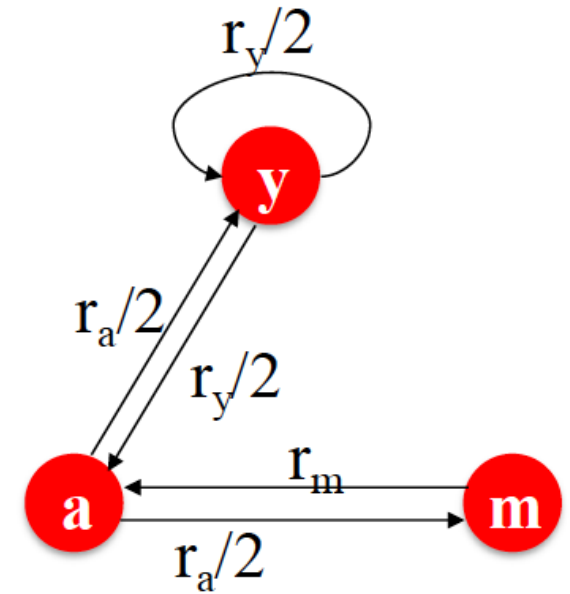


PageRank: The "Flow" Model

- A page is important if it is pointed to by other important pages
- Define rank r_j for node j :

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

d_i : out-degree of node i



“Flow” equations:

$$r_y = r_y/2 + r_a/2$$

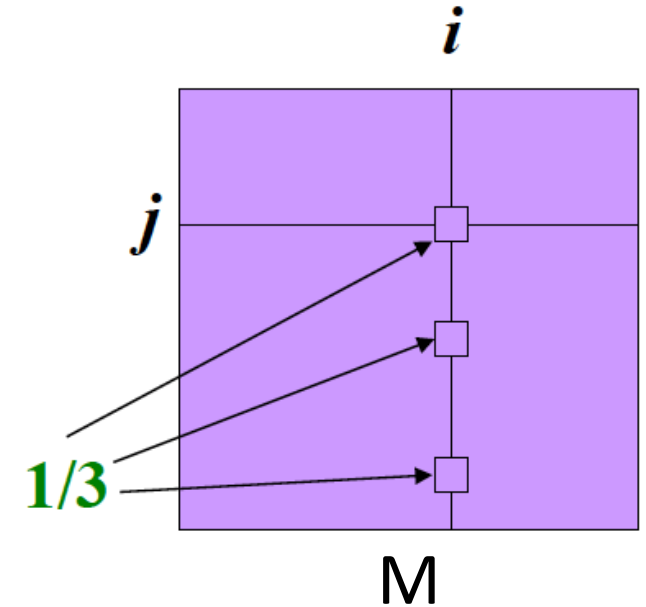
$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

PageRank: Matrix Formulation

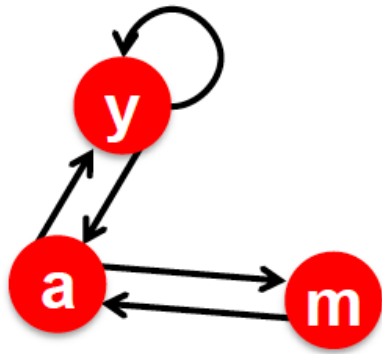
- **Stochastic adjacency matrix M**
 - d_i : out-degree of node i
 - If $i \rightarrow j$, then $M_{ij} = \frac{1}{d_i}$
 - M is column stochastic matrix
 - Columns sum to 1.
- **Rank vector r** : an entry per page
 - r_i : the importance score of page i
 - $\sum_i r_i = 1$
- **Flow equations:**

$$r = M \cdot r$$



$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

Example: Flow Equations and M



	r_y	r_a	r_m
r_y	$\frac{1}{2}$	$\frac{1}{2}$	0
r_a	$\frac{1}{2}$	0	1
r_m	0	$\frac{1}{2}$	0

$$r_y = r_y / 2 + r_a / 2$$

$$r_a = r_y / 2 + r_m$$

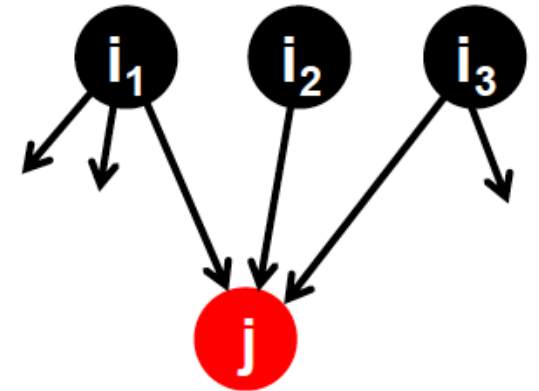
$$r_m = r_a / 2$$

$$\begin{matrix} r_y \\ r_a \\ r_m \end{matrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 1 \\ 0 & \frac{1}{2} & 0 \end{bmatrix} \begin{matrix} r_y \\ r_a \\ r_m \end{matrix}$$

$r \qquad M \qquad r$

Connection to Random Walk

- Imagine a random web surfer:
 - At any time t , surfer is on some page i
 - At time $t+1$, surfer follows an out-link from i uniformly at random
 - Ends up on some page j linked to i
 - Process repeats indefinitely
- Let:
 - $\mathbf{p}(t)$: a vector --- i th coordinate is prob. the surfer is at page i at time t
 - Essentially, $\mathbf{p}(t)$ is the probability distribution over pages



$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

Connection to Random Walk: Stationary Distribution

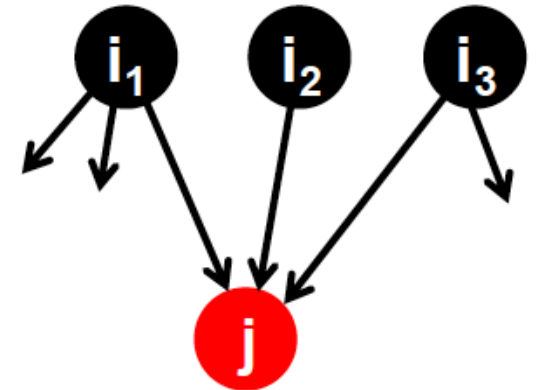
- Where is suffer at time $t+1$?

$$p(t + 1) = M \cdot p(t)$$

- Suppose the random walk reaches a state

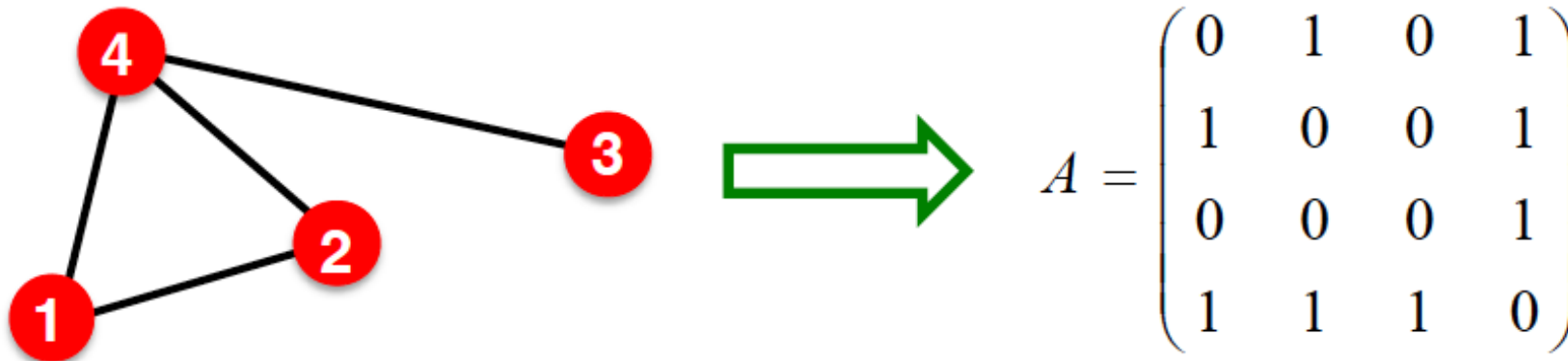
$$p(t + 1) = M \cdot p(t) = p(t)$$

- Then $p(t)$ is **stationary distribution** of random walk
- Our original rank vector r satisfies: $r = M \cdot r$
 - It means **r is stationary distribution of random walk**



Recall: Eigenvector of a matrix

- Adjacency matrix of undirected graph



- Eigenvector of adjacency matrix: $\lambda c = Ac$
 - c : eigenvector, λ : eigenvalue
- Note:
 - This is definition of eigenvector centrality (for undirected graph)
 - PageRank is defined for directed graphs

Eigenvector Formulation

- The flow equation: $1 \cdot r = M \cdot r$

$$\begin{array}{|c|} \hline r_y \\ \hline r_a \\ \hline r_m \\ \hline \end{array} = \begin{array}{|ccc|} \hline \frac{1}{2} & \frac{1}{2} & 0 \\ \hline \frac{1}{2} & 0 & 1 \\ \hline 0 & \frac{1}{2} & 0 \\ \hline \end{array} \begin{array}{|c|} \hline r_y \\ \hline r_a \\ \hline r_m \\ \hline \end{array}$$

$\mathbf{r} \qquad \mathbf{M} \qquad \mathbf{r}$

- So the rank vector r is an eigenvector of the stochastic adjacency matrix M (with eigenvalue 1)
 - Starting from any vector u , the limit $M \left(M \left(\dots M(M(u)) \right) \right)$ is the long-term distribution of the surfer.
 - PageRank (r): limiting distribution = the principal eigenvector of M
- We can now efficiently solve for r
 - The method is called Power iteration

PageRank: Summary

- Measure importance of nodes in a graph using the link structure of the web
- Model a random web surfer using stochastic adjacency matrix M
- Solve $r = Mr$ where r can be viewed as both the principal eigenvector of M and as the stationary distribution of a random walk over the graph

PageRank: How to Solve?

- Given a graph with n nodes, we use an iterative procedure
 - Assign each node an initial page rank
 - Repeat until convergence: $\sum_i |r_i^{t+1} - r_i^t| < \epsilon$
 - Calculate the page rank of each node:

$$r_j^{t+1} = \sum_{i \rightarrow j} \frac{r_i^t}{d_i}$$

- d_i : out-degree of node i

Power Iteration Method

- Given a web graph with N nodes, where nodes are pages and edges are hyperlinks
- **Power iteration**: a simple iterative scheme
 - Initialize: $r^{(0)} = \left[\frac{1}{N}, \frac{1}{N}, \dots, \frac{1}{N}\right]^T$
 - Iterate: $r^{(t+1)} = M \cdot r^{(t)}$
 - Stop when: $|r^{(t+1)} - r^{(t)}|_1 < \epsilon$
- Note: About 50 iterations is sufficient to estimate the limiting solution

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

PageRank: Three Questions

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

Or equivalently,

$$r = Mr$$

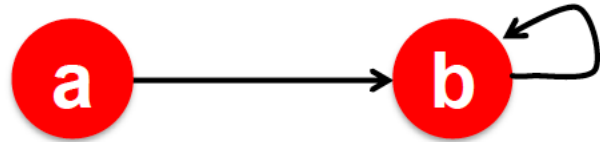
- Does this converge?
- Does it converge to what we want?
- Are results reasonable?

PageRank: Problems

- Two problems:
 1. Some pages are **dead ends** (have no out-links)
 - These pages cause importance to leak out
 2. **Spider traps** (all out-links are within the group)
 - Eventually, spider traps absorb all importance

Does This Converge?

- The **spider trap** problem:



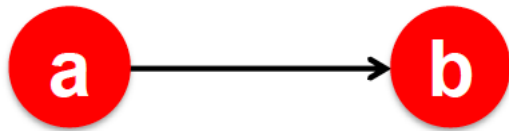
$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

- PageRank iteration

Iteration	0	1	2	3
r_a	1	0	0	0
r_b	0	1	1	1

Does It Converge to What We Want?

- The **dead-end** problem:



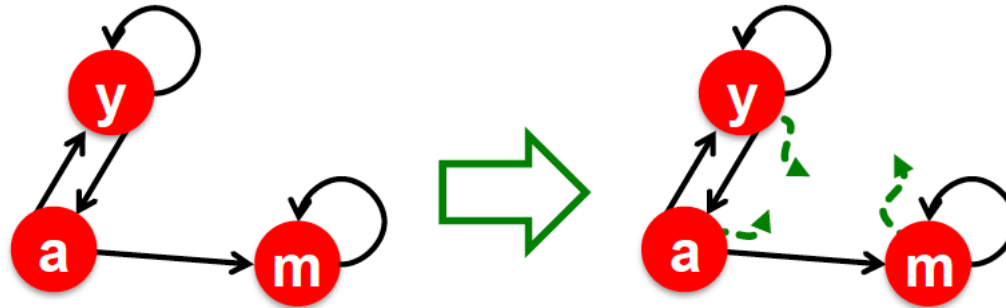
$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

- PageRank iteration

Iteration	0	1	2	3
r_a	1	0	0	0
r_b	0	1	0	0

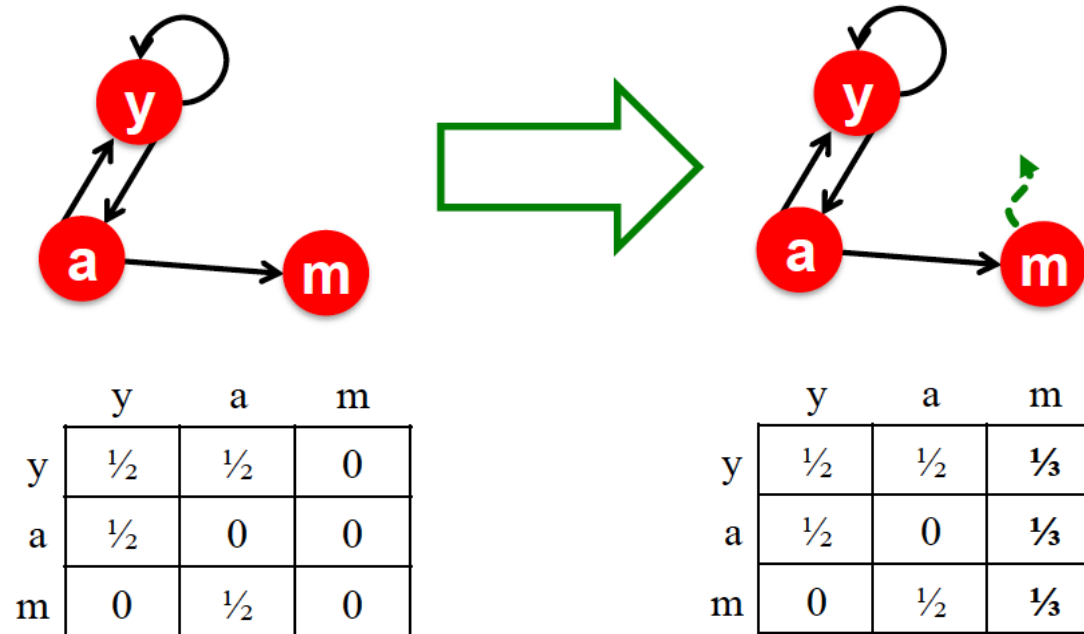
Solution to Spider Traps

- At each time step, random surfer has two options
 - With prob. β , follow a link at random
 - With prob. $1 - \beta$, jump to a random page
 - Common values for β : $[0.8, 0.9]$
- Surfer will teleport out of spider trap within a few steps



Solution to Dead Ends

- Teleports: Follow random teleport links with total probability 1.0 from dead-ends
 - Adjust adjacency matrix accordingly



Why Teleports Solve the Problem

- Why are dead-ends and spider traps a problem and why do teleports solve the problem?
 - **Spider-traps** are not a problem, but with traps PageRank scores are not what we want.
 - **Solution**: Never get stuck in a spider trap by teleporting out of it in a finite number of steps
 - **Dead-ends** are a problem
 - The matrix is not column stochastic so our initial assumptions are not met
 - **Solution**: Make matrix column stochastic by always teleporting when there is nowhere to go

Solution: Random Teleports

- Google's solution does it all
- At each step, random surfer has two options
 - With probability β , follow a link at random
 - With probability $1 - \beta$, jump to some random page
- PageRank equation:

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \beta \frac{r_i^{(t)}}{d_i} + (1 - \beta) \frac{1}{N}$$

The Google Matrix

- PageRank equation:

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \beta \frac{r_i^{(t)}}{d_i} + (1 - \beta) \frac{1}{N}$$

- The Google matrix

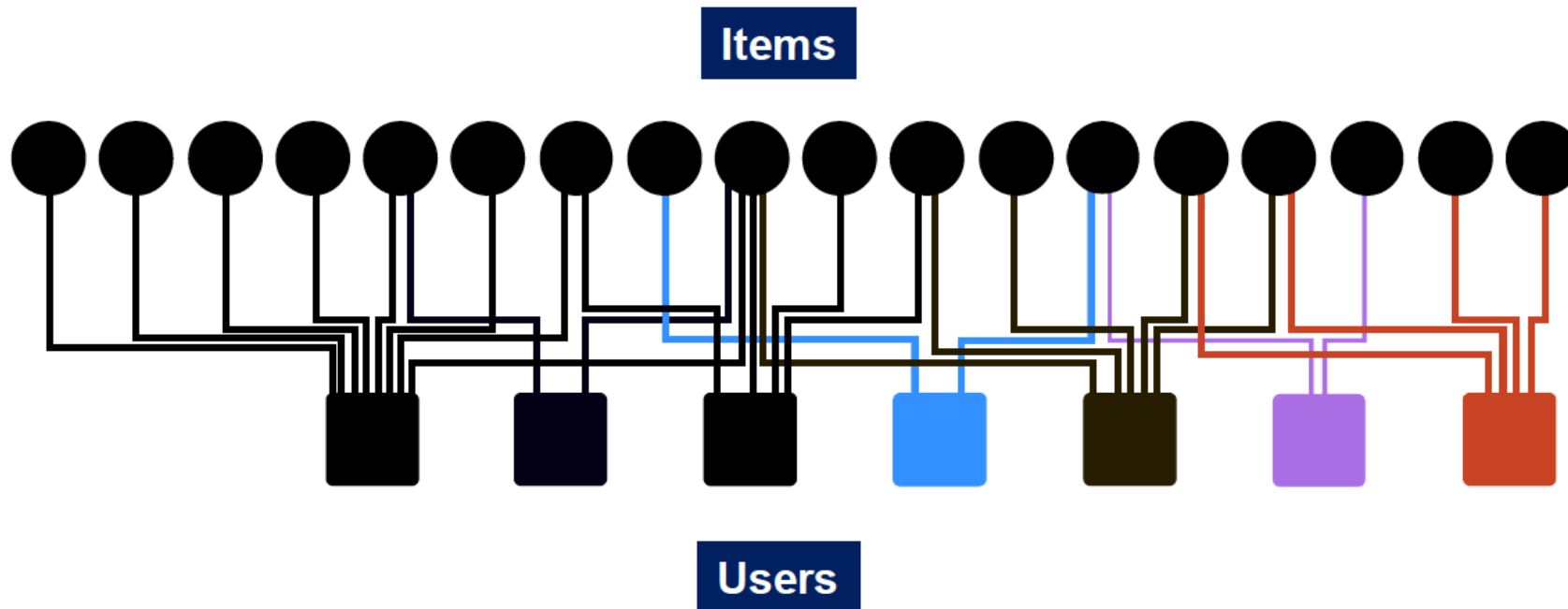
$$G = \beta M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$$

- We have a recursive problem: $r = Gr$
 - The Power method still works
 - In practice, $\beta \in [0.8, 0.9]$

Random Walk with Restarts and Personalized PageRank

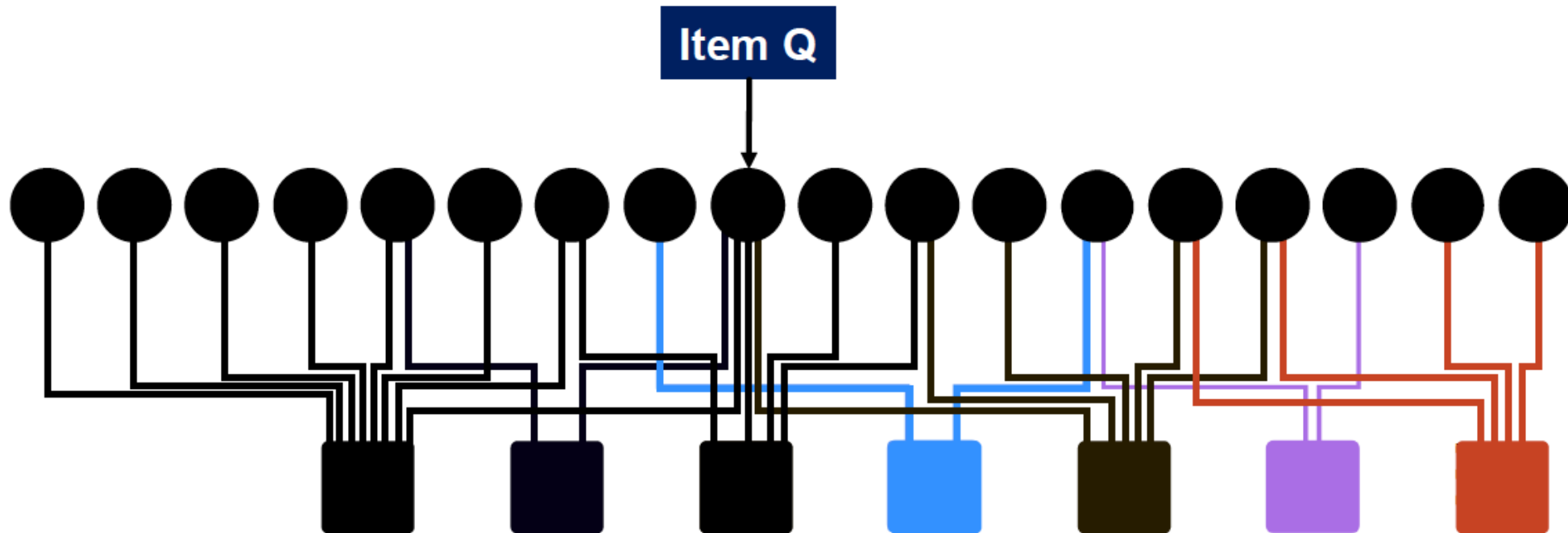
Example: Recommendation

- Given: a bipartite graph representing user and item interactions



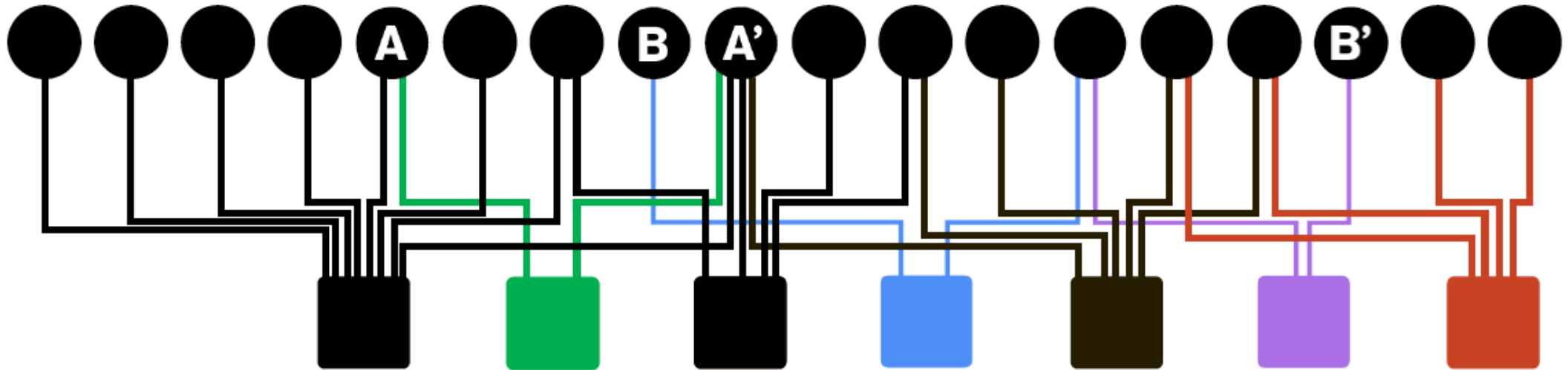
Bipartite User-Item Graph

- **Goal:** proximity on graphs
 - What items should we recommend to a user who interacts with item Q?
 - **Intuition:** if items Q and P are interacted by similar users, recommend P to users interacts with Q



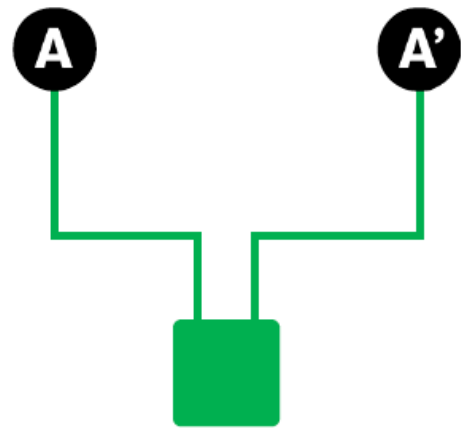
Bipartite User-Item Graph

- Which is more related? A, A' or B, B'

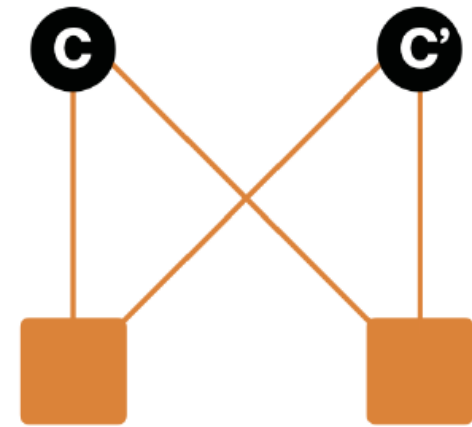
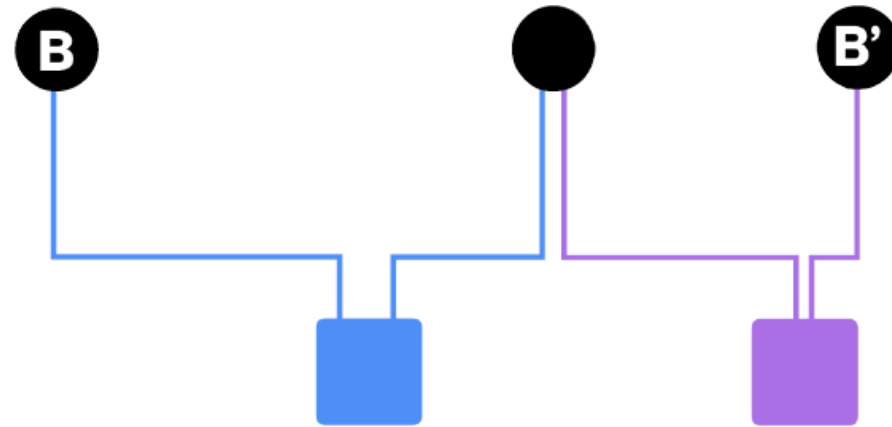


Node Proximity Measurements

- Which is more related A, A' or B, B' or C, C'?



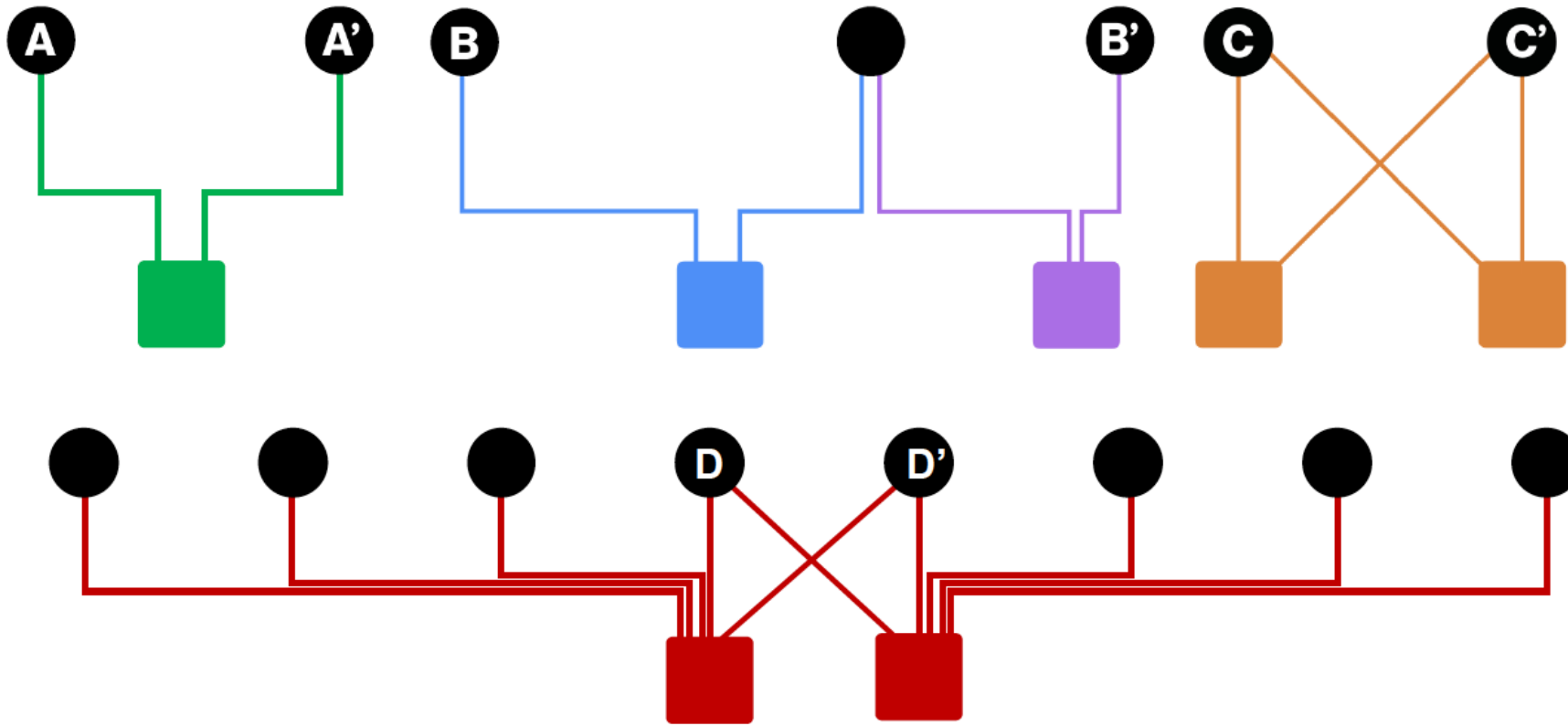
Shortest path



Common Neighbors

Node Proximity Measurements

- Which is more related A, A' or B, B' or C, C'?



Personalized Page Rank/Random Walk with Restarts

Proximity on Graphs

- PageRank:
 - Rank nodes by importance
 - Teleports with uniform probability to any node in network
- Personalized PageRank
 - Ranks proximity of nodes to the teleport nodes S
- Proximity on graphs
 - Q: What is most related item to item Q ?
 - Random Walks with Restarts
 - Teleport back to the starting node $S = \{Q\}$

Idea: Random Walks

- **Idea:**

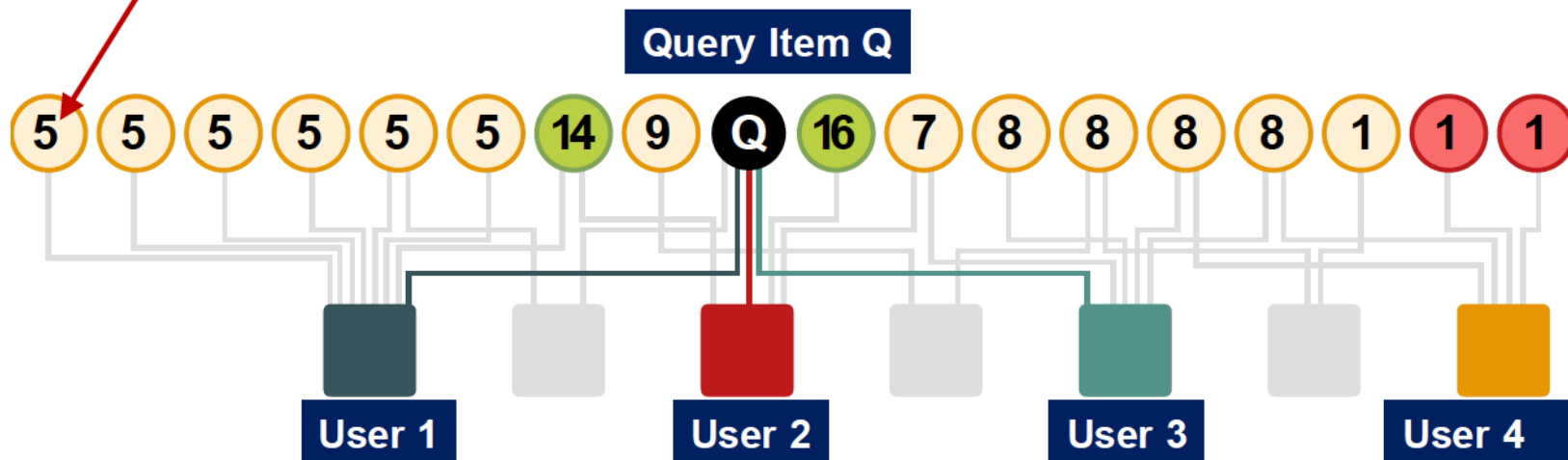
- Every node has some importance
- Importance gets evenly split among all edges and pushed to neighbors

- Given a set of QUERY_NODES, we simulate a random walk
 1. Make a step to a random neighbor and record the visit (visit count)
 2. With probability α , restart the walk at one of the query nodes in QUERY_NODES
 3. The nodes with highest visit count have the highest proximity to QUERY_NODES

Random Walk Algorithms

```
item = QUERY_NODES.sample_by_weight( )  
for i in range( N_STEPS ):  
    user = item.get_random_neighbor( )  
    item = user.get_random_neighbor( )  
    item.visit_count += 1  
    if random( ) < ALPHA:  
        item = QUERY_NODES.sample.by_weight ( )
```

Number of
random visits



Benefits

- Why is this a good solution?
- Because the similarity considers:
 - Multiple connections
 - Multiple paths
 - Direct and undirect connects
 - Degree of the nodes

Summary: PageRank Variants

■ PageRank

- Teleport to any nodes
- Nodes have the same probability of the suffer landing

$$\mathbf{S} = [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]$$

■ Personalized PageRank

- Teleport to a specific set of nodes
- Nodes can have different probabilities of suffer landing

$$\mathbf{S} = [0.1, 0, 0, 0.2, 0, 0, 0.5, 0, 0, 0.2]$$

■ Random Walk with Restarts

- Teleport is always to the same node

$$\mathbf{S} = [0, 0, 0, 0, \mathbf{1}, 0, 0, 0, 0, 0]$$

Summary

- A graph is naturally represented as a matrix
- We defined a random walk process over the graph
 - Random surfer moving across the links and with random teleportation
 - Stochastic adjacency matrix M
- PageRank = Limiting distribution of the surfer location represented node importance
 - Corresponds to the leading eigenvector of transformed adjacency matrix M